المدرسة الوطنية للعلوم التطبيقية -أكادير
ⵜⴰⵙⴷⴰⵡⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵣⵔⴼⵜ ⵜⵉⵏⵖⵉⵔⵉⵏ- ⴰⴳⴰⴷⵉⵔ.
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES - AGADIR

ENSA-A/FID3/2026-2027

Filière : **Finance et Ingénierie Décisionnelle**

Les modèles binomial et trinomial appliqués à la valorisation des options

Module : **Valorisation et Couverture par les Produits Dérivés**

Réalisé par :
Tildi Loubna

Encadré par :
Pr. Brahim EL ASRI

13 octobre 2026

Résumé

Ce projet prolonge nos travaux des deux premières années du cycle, consacrés à la valorisation d'options européennes par la formule de Black–Scholes et par simulation de Monte-Carlo. Nous étudions ici deux **modèles d'arbres** : le **modèle binomial** de Cox–Ross–Rubinstein, où le sous-jacent peut monter ou baisser à chaque pas, et le **modèle trinomial**, qui ajoute un troisième état « stable ». Ces modèles reconstruisent le prix d'une option européenne par récurrence rétrograde sur un arbre de prix, et leur limite lorsque le pas de temps tend vers zéro est le modèle de Black–Scholes. Nous montrons que le passage de deux à trois états a une conséquence théorique importante : le marché trinomial est *incomplet*, la probabilité risque-neutre n'y est plus unique et la réplication parfaite n'est plus possible, ce qui oblige à *calibrer* l'arbre sur la volatilité. Les deux modèles sont implémentés en Python, validés (exemple du cours, parité call–put, lien exact entre les deux arbres) et comparés : convergence vers Black–Scholes, oscillations, précision à temps de calcul égal, sensibilités et Greeks. Le trinomial supprime les oscillations « pair/impair » du binomial et, bien paramétré, donne une erreur environ trois fois plus faible pour le même nombre de pas. Une interface web interactive permet enfin d'explorer le modèle binomial et de le confronter à des prix de marché réels.

Mots-clés : Modèle binomial, Modèle trinomial, Options européennes, Réplication, Marché incomplet, Probabilité risque-neutre, Black–Scholes, Convergence.

Abstract

This project extends our previous work on European option pricing with the Black–Scholes formula and Monte Carlo simulation. We study two **lattice models**: the Cox–Ross–Rubinstein **binomial model**, where the underlying moves up or down at each step, and the **trinomial model**, which adds a third "flat" state. Both models price European options by backward induction on a price tree, and their continuous-time limit is the Black–Scholes model. Moving from two to three states has an important theoretical consequence: the trinomial market is *incomplete*, the risk-neutral probability is no longer unique and perfect replication is impossible, so the tree must be *calibrated* to volatility. Both models are implemented in Python, validated (course example, put–call parity, exact link between the two trees) and compared: convergence towards Black–Scholes, oscillations, accuracy at equal computing time, sensitivities and Greeks. The trinomial tree removes the odd/even oscillations of the binomial tree and, when well parameterised, is about three times more accurate for the same number of steps. An interactive web interface lets the reader explore the binomial model and compare it with real market prices.

Keywords: Binomial model, Trinomial model, European options, Replication, Incomplete market, Risk-neutral probability, Black–Scholes, Convergence.

Table des matières

Introduction générale **1**

Chapitre 1 : Notions de base sur les options **2**
1.1 Call, put et payoff 2
1.2 Options européennes et américaines 2
1.3 Taux sans risque et volatilité 2
1.4 Absence d'opportunité d'arbitrage 3
1.5 Récapitulatif des notations 3

Chapitre 2 : Le modèle binomial **4**
2.1 Présentation 4
2.2 Hypothèses et principe 4
2.3 L'arbre à une période 4
2.4 L'arbre à N périodes 5
2.5 Delta et couverture dynamique 6
2.6 Calibrage de u et d sur la volatilité (modèle CRR) 6

Chapitre 3 : Le modèle trinomial **7**
3.1 De deux à trois états 7
3.2 Un marché incomplet 7
3.3 Calibrage de l'arbre trinomial sur la volatilité 8
3.4 Lien exact avec le binomial : deux pas binomiaux font un pas trinomial 9
3.5 Valorisation par récurrence rétrograde 9
3.6 Couverture et Greeks dans l'arbre trinomial 10
3.7 Interprétation : un schéma aux différences finies 10
3.8 Synthèse : binomial ou trinomial ? 10

Chapitre 4 : Convergence vers Black–Scholes **11**
4.1 Rappel : la formule de Black–Scholes 11
4.2 Énoncé du résultat 11
4.3 Pourquoi le prix converge 11
4.4 Vitesse de convergence et oscillations 12

Chapitre 5 : Implémentation **13**
5.1 Choix techniques et architecture 13
5.2 Algorithmes 13
5.3 Le cœur du code 13
5.4 Greeks et volatilité implicite 14
5.5 Validation par tests automatiques 14
5.6 Interface interactive 14

Chapitre 6 : Résultats et comparaison des deux modèles **15**
6.1 Convergence du binomial vers Black–Scholes 15
6.2 Convergence du trinomial et comparaison 16
6.3 Rôle du paramètre λ 17
6.4 Précision à temps de calcul égal 18
6.5 Sensibilité aux paramètres 19
6.6 Delta, gamma et couverture 19
6.7 Confrontation au marché réel 20

Chapitre 7 : Limites et extensions **21**

7.1 Limites 21
7.2 Extensions 21

Conclusion générale **22**

Annexe A : Captures de l'interface **23**

Annexe B : Code Python du modèle binomial **26**

Annexe C : Code Python du modèle trinomial **28**

Bibliographie **30**

Introduction générale

Les produits dérivés occupent une place centrale dans les marchés financiers modernes. Parmi eux, les options permettent à la fois de se couvrir contre un risque (de prix, de taux ou de change) et de prendre une position sur l'évolution d'un actif. Leur utilisation suppose de savoir leur attribuer un prix : c'est l'objet de la valorisation, qui est au cœur de la finance de marché et de la gestion des risques.

Valoriser une option ne consiste pas à prédire l'avenir du sous-jacent, mais à déterminer le prix cohérent avec l'absence d'opportunité d'arbitrage, c'est-à-dire le coût d'une stratégie qui reproduit exactement le gain de l'option. Plusieurs méthodes existent ; elles diffèrent par leurs hypothèses, par leur précision et par les types d'options qu'elles savent traiter.

Ce projet s'inscrit dans la continuité de nos travaux des deux premières années du cycle, consacrés à la valorisation d'options européennes par la formule de Black–Scholes et par simulation de Monte-Carlo. Nous y ajoutons cette année les **modèles d'arbres**. Le **modèle binomial**, vu en cours, montre comment le prix se construit par réplication, étape par étape, sur un arbre de prix. Le **modèle trinomial** en est la généralisation naturelle : à chaque pas, le sous-jacent peut monter, baisser *ou rester stable*. Ce troisième état rend l'arbre plus souple et, comme nous le verrons, plus régulier numériquement, mais il change aussi la nature du marché : on ne peut plus tout répliquer. La question qui guide ce travail est donc la suivante : *Comment les modèles binomial et trinomial permettent-ils de valoriser des options européennes, que change le passage de deux à trois états, et lequel des deux converge le mieux vers le prix de Black–Scholes ?*

Le rapport est organisé comme suit. Le chapitre 1 rappelle les notions de base sur les options. Le chapitre 2 présente le modèle binomial et le chapitre 3 le modèle trinomial, en insistant sur ce qui change entre les deux. Le chapitre 4 établit le lien avec Black–Scholes. Le chapitre 5 décrit l'implémentation, le chapitre 6 présente et compare les résultats, et le chapitre 7 discute les limites et les extensions avant la conclusion.

Chapitre 1. Notions de base sur les options

1.1 Call, put et payoff

Une *option* est un contrat qui donne le droit, sans l'obligation, d'acheter ou de vendre un actif à un prix fixé à l'avance. Dans ce rapport, nous traitons les options dites **vanilles**, c'est-à-dire standard : les *calls* et les *puts* **européens**. On les oppose aux options *exotiques* (à barrière, asiatiques, digitales, etc.), dont le payoff est plus complexe.

Un **call** donne le droit d'*acheter* un actif (le *sous-jacent*, de prix S) au prix d'exercice K (le *strike*) ; un **put** donne le droit de le *vendre* au prix K. La date limite est la *maturité* T. À l'échéance, le détenteur n'exerce que si cela lui rapporte : le gain, ou *payoff*, vaut

$$h_{\text{call}}(S_T) = (S_T - K)^+ = \max(S_T - K, 0), \qquad h_{\text{put}}(S_T) = (K - S_T)^+ = \max(K - S_T, 0).$$

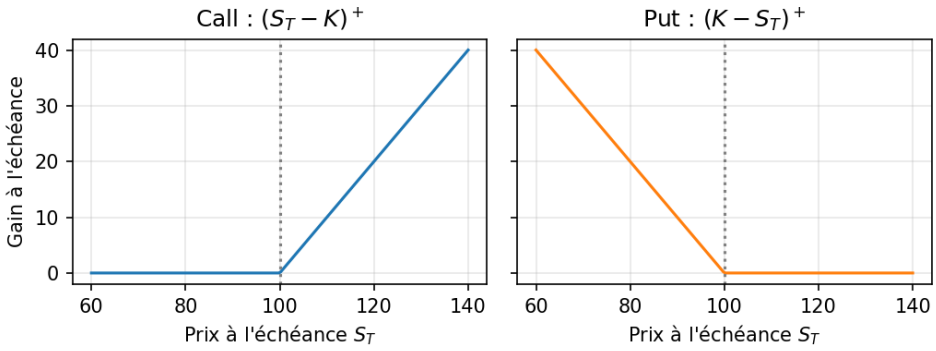


Figure 1: Payoffs d'un call et d'un put de strike $K = 100$.

Exemple. Un call de strike 100 sur une action qui vaut 120 à l'échéance rapporte 20 ; si l'action vaut 90, il ne rapporte rien (on ne perd que la prime payée à l'achat). Le payoff ne dit pas combien l'option vaut *aujourd'hui* : c'est l'objet de la valorisation.

1.2 Options européennes et américaines

Une option **européenne** ne peut être exercée qu'à la maturité T. Une option **américaine** peut l'être à *toute date* $t \leq T$. Ce rapport se limite aux options **européennes**, pour lesquelles Black–Scholes fournit un prix exact qui sert de référence. Les options américaines, qui demandent la théorie de l'arrêt optimal (chapitre 4 du cours), sont mentionnées en perspective (chapitre 7).

1.3 Taux sans risque et volatilité

Le **taux sans risque** r est le rendement d'un placement sans risque (par exemple un bon du Trésor). Dans le cours, on note $R = 1 + r$ le facteur de capitalisation sur une période ; avec un taux continu on a $R = e^{r\Delta t}$. Un euro placé aujourd'hui vaut R à la période suivante, et un euro reçu dans une période ne vaut que $1/R$ aujourd'hui (c'est l'*actualisation*).

La **volatilité** σ mesure l'amplitude des fluctuations du prix du sous-jacent (écart-type annualisé de ses rendements). Plus σ est élevée, plus l'issue est incertaine et plus une option vaut cher, car son détenteur profite des mouvements favorables sans subir les défavorables.

1.4 Absence d'opportunité d'arbitrage

Une *opportunité d'arbitrage* est une stratégie qui ne coûte rien, ne peut pas faire perdre d'argent, et rapporte un gain avec une probabilité strictement positive. Le principe de tout le cours est que, sur un marché raisonnable, de telles stratégies n'existent pas. Conséquence immédiate : deux portefeuilles qui produisent exactement les mêmes flux futurs ont le même prix aujourd'hui. C'est ce principe qui permet de fixer le prix d'une option en la *répliquant*.

La **parité call–put** en est une illustration. Pour des options européennes de même strike K et de même maturité, sans dividendes,

$$C - P = S_0 - \frac{K}{R^N}.$$

En effet, acheter un call et vendre un put donne le payoff $(S_T - K)^+ - (K - S_T)^+ = S_T - K$, identique à celui d'un portefeuille « une action, moins K euros empruntés jusqu'à T ». Deux portefeuilles de même payoff ont le même prix. Cette relation ne dépend d'*aucun* modèle : elle sera vérifiée aussi bien dans l'arbre binomial que dans l'arbre trinomial.

1.5 Récapitulatif des notations

S_0, S_k	prix du sous-jacent en $t = 0$ et au pas k
K, T, N	strike, maturité, nombre de pas ($\Delta t = T/N$)
r, R	taux sans risque ; facteur de capitalisation par période ($R = 1 + r$ ou $e^{r\Delta t}$)
σ	volatilité annualisée
u, d	facteurs de hausse et de baisse ($0 < d < u$)
q	probabilité risque-neutre de hausse (binomial)
q_u, q_m, q_d	probabilités risque-neutres de hausse, de stabilité et de baisse (trinomial)
λ	paramètre d'écartement de l'arbre trinomial ($u = e^{\lambda\sigma\sqrt{\Delta t}}$)
$h(\cdot)$	payoff ($(S-K)^+$ pour un call, $(K-S)^+$ pour un put)
$V_k(S)$	valeur de l'option au pas k quand le sous-jacent vaut S
Δ	quantité de sous-jacent dans le portefeuille de couverture (delta)

Chapitre 2: Le modèle binomial

2.1 Présentation

Le modèle binomial a été proposé par Cox, Ross et Rubinstein en 1979 [4]. C'est un modèle *en temps discret* : le prix du sous-jacent évolue par pas successifs et, à chaque pas, il ne peut prendre que deux valeurs. Le modèle de Black–Scholes en est la limite lorsque la durée des pas tend vers zéro (chapitre 4). Ses principales caractéristiques sont les suivantes :

- il repose uniquement sur la **réplication** et l'**absence d'arbitrage**, sans recours au calcul stochastique ;
- il donne à chaque nœud le **portefeuille de couverture** (le delta) ;
- il est simple à programmer : le prix s'obtient par une récurrence rétrograde sur un arbre ;
- il **converge vers Black–Scholes** quand le nombre de pas N augmente ;
- en contrepartie, son coût de calcul croît en N^2 et sa convergence est lente (erreur en $1/N$) et *oscillante*.

2.2 Hypothèses et principe

Le temps est découpé en N périodes de durée $\Delta t = T/N$. À chaque période, le prix du sous-jacent est multiplié soit par u (hausse), soit par d (baisse), avec $0 < d < u$:

$$S_{k+1} = S_k \cdot u \quad \text{ou} \quad S_{k+1} = S_k \cdot d.$$

On suppose qu'il existe un actif sans risque de facteur de capitalisation R par période, et que le marché est sans arbitrage et sans frais de transaction.

2.3 L'arbre à une période

La réplication

Considérons une option de payoff $h(S_1)$ en $t = 1$. On cherche un portefeuille composé de Δ unités de sous-jacent et d'une somme B placée (ou empruntée si $B < 0$) au taux sans risque, qui *reproduit exactement* le payoff dans les deux états. Sa valeur initiale est $x = \Delta S_0 + B$ et sa valeur en $t = 1$ est $\Delta S_1 + RB$. On impose

$$\begin{cases} \Delta\, uS_0 + RB = h(uS_0) \\ \Delta\, dS_0 + RB = h(dS_0) \end{cases}$$

Deux équations, deux inconnues : le système a une solution unique. En soustrayant les deux lignes on obtient

$$\boxed{\Delta = \frac{h(uS_0) - h(dS_0)}{(u-d)\,S_0}}$$

puis B s'en déduit. Par **absence d'arbitrage**, le prix de l'option est la valeur initiale du portefeuille, $x = \Delta S_0 + B$. Un calcul direct donne

$$\boxed{x = \frac{1}{R}\Big[q\,h(uS_0) + (1-q)\,h(dS_0)\Big], \qquad q = \frac{R-d}{u-d}.}$$

Comme toute option est ainsi duplicable, on dit que le marché binomial est **complet**. Le cours en déduit l'**unicité de la probabilité risque-neutre** (proposition 2.4.1 du cours [1]) : c'est précisément cette propriété qui disparaîtra dans le modèle trinomial.

Condition de non-arbitrage et probabilité risque-neutre

Le nombre q est une probabilité (c'est-à-dire $0 < q < 1$) si et seulement si

$$d < R < u.$$

Si $R \geq u$, l'actif sans risque rapporte au moins autant que l'actif risqué dans les deux états : en vendant l'actif risqué et en plaçant le produit, on gagne sans risque, c'est un arbitrage. Si $R \leq d$, c'est l'inverse. La condition $d < R < u$ est donc exactement l'absence d'arbitrage.

La probabilité q est dite *risque-neutre*. Elle vérifie

$$\mathbb{E}_{\mathbb{Q}}[S_1] = q\,uS_0 + (1-q)\,dS_0 = R\,S_0,$$

c'est-à-dire que, sous q, le prix actualisé S_k/R^k est une *martingale*. Le prix de l'option s'écrit alors comme une espérance actualisée : $x = \mathbb{E}_{\mathbb{Q}}[h(S_1)]/R$.

Point clé : q n'est pas la vraie probabilité de hausse

q ne dépend que de u, d et R. Même si la vraie probabilité de hausse valait 0,9 ou 0,1, le prix de l'option serait le même, car il est imposé par la réplication et l'arbitrage, pas par les anticipations. q est un poids de calcul qui rend le prix actualisé martingale.

Exemple chiffré du cours

Prenons $S_0 = 100$, $K = 100$, $R = 1{,}05$ (soit $r = 5\,\%$), $u = 1{,}1$, $d = 0{,}9$. Alors $q = \frac{1{,}05-0{,}9}{1{,}1-0{,}9} = 0{,}75$.

Call. Les payoffs sont $h(110) = 10$ et $h(90) = 0$. Le système s'écrit $110\Delta + 1{,}05B = 10$ et $90\Delta + 1{,}05B = 0$. En soustrayant : $20\Delta = 10$, donc $\Delta = 0{,}5$; puis $1{,}05B = -45$, donc $B \approx -42{,}857$. Le portefeuille de réplication consiste à acheter 0,5 action en empruntant 42,857. Sa valeur initiale est $0{,}5 \times 100 - 42{,}857 \approx \mathbf{7{,}14}$, ce qui est le prix du call. *Vérification :* si le prix monte à 110, le portefeuille vaut $0{,}5 \times 110 - 42{,}857 \times 1{,}05 = 55 - 45 = 10$; s'il baisse à 90, il vaut $45 - 45 = 0$. Il reproduit bien le payoff. On retrouve aussi $\frac{0{,}75\times 10}{1{,}05} \approx 7{,}14$.

Put. Les payoffs sont $h(110) = 0$ et $h(90) = 10$. On trouve $\Delta = -0{,}5$ (vente à découvert d'une demi-action) et un prix $\frac{0{,}25\times 10}{1{,}05} \approx \mathbf{2{,}38}$.

Parité. $C - P = 7{,}14 - 2{,}38 = 4{,}76 = 100 - \frac{100}{1{,}05}$: la parité call–put est vérifiée.

2.4 L'arbre à N périodes

Un arbre recombinant

On répète le mécanisme précédent à chaque période. Comme une hausse suivie d'une baisse donne le même prix qu'une baisse suivie d'une hausse ($ud = du$), l'arbre est *recombinant* : au pas k il n'y a que $k+1$ nœuds, et le nœud ayant subi j hausses vaut

$$S_{k,j} = S_0\,u^j\,d^{\,k-j}, \qquad j = 0, \ldots, k.$$

La récurrence rétrograde (backward induction)

À maturité, la valeur de l'option est son payoff : $V_N(S) = h(S)$. On remonte ensuite l'arbre, en appliquant en chaque nœud le résultat de la période unique :

$$V_k(S) = \frac{1}{R}\Big[q\,V_{k+1}(uS) + (1-q)\,V_{k+1}(dS)\Big].$$

Cette récurrence équivaut, pour une option européenne, à la formule

$$V_0 = \frac{1}{R^N}\,\mathbb{E}_{\mathbb{Q}}\big[h(S_N)\big] = \frac{1}{R^N}\sum_{j=0}^{N}\binom{N}{j} q^j (1-q)^{N-j}\, h\big(S_0 u^j d^{N-j}\big),$$

car le nombre de hausses suit une loi binomiale de paramètres (N, q) sous la probabilité risque-neutre. La parité call–put du chapitre précédent en découle puisque $\mathbb{E}_{\mathbb{Q}}[S_N] = R^N S_0$.

Exemple à trois périodes

Avec les mêmes paramètres ($S_0 = K = 100$, $u = 1{,}1$, $d = 0{,}9$, $R = 1{,}05$, $q = 0{,}75$), la figure 2 donne l'arbre d'un *call européen*. Par exemple, au nœud $S = 121$ du pas 2, $V = \frac{0{,}75\times 33{,}1 + 0{,}25\times 8{,}9}{1{,}05} \approx 25{,}76$. Le prix en $t = 0$ est **15,31**.

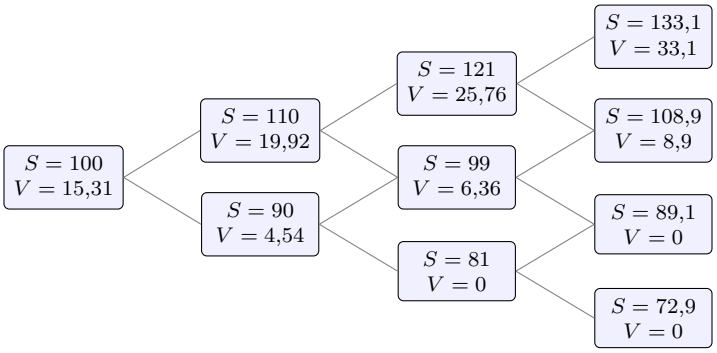


Figure 2: Call européen de strike 100 sur 3 périodes. À maturité, $V = (S-K)^+$; on remonte avec $V = \frac{1}{R}[qV^{\uparrow} + (1-q)V^{\downarrow}]$.

2.5 Delta et couverture dynamique

Le portefeuille de réplication est reconstitué à chaque nœud : la quantité de sous-jacent à détenir est

$$\Delta_{k,j} = \frac{V_{k+1}(uS) - V_{k+1}(dS)}{(u-d)\,S}, \qquad S = S_{k,j}.$$

Un vendeur d'option se couvre en détenant Δ unités de sous-jacent et en ajustant cette quantité à chaque période. Pour le call de la figure 2, $\Delta_0 \approx 0{,}77$; au pas 1 il vaut 0,35 si le prix a baissé et 0,88 s'il a monté : la couverture s'adapte au prix. Dans le modèle binomial, cette couverture est **parfaite** : elle élimine tout risque. C'est cette information que l'interface affiche lorsqu'on survole un nœud.

2.6 Calibrage de u et d sur la volatilité (modèle CRR)

Le cours laisse u et d comme paramètres libres. Pour relier l'arbre à un sous-jacent de volatilité σ et à un taux continu r, Cox, Ross et Rubinstein (1979) posent

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u}, \qquad R = e^{r\Delta t}, \qquad q = \frac{e^{r\Delta t} - d}{u - d}.$$

L'idée est que, sur un pas, le rendement logarithmique vaut $\pm\sigma\sqrt{\Delta t}$; un développement limité donne $q \approx \frac{1}{2} + \frac{(r-\sigma^2/2)\sqrt{\Delta t}}{2\sigma}$, de sorte que le rendement logarithmique moyen par pas vaut $\approx (r - \sigma^2/2)\Delta t$ et sa variance $\approx \sigma^2 \Delta t$. L'arbre imite ainsi un mouvement brownien géométrique, ce qui explique la convergence vers Black–Scholes (chapitre 4).

Chapitre 3: Le modèle trinomial

3.1 De deux à trois états

Dans le modèle binomial, le sous-jacent ne peut que monter ou baisser à chaque pas. Le **modèle trinomial**, introduit par Boyle en 1986 [5], ajoute un troisième mouvement : le prix peut aussi **rester stable**. À chaque période,

$$S_{k+1} = S_k \cdot u \quad \text{ou} \quad S_{k+1} = S_k \cdot m \quad \text{ou} \quad S_{k+1} = S_k \cdot d, \qquad 0 < d < m < u.$$

Dans toute la suite on prend $m = 1$ et $d = 1/u$, ce qui rend l'arbre *recombinant* : une hausse suivie d'une baisse ramène au même prix que deux pas stables. Au pas k, l'arbre compte $2k+1$ nœuds (contre $k+1$ pour le binomial), et le nœud d'indice j vaut

$$S_{k,j} = S_0\, u^j, \qquad j = -k, \dots, k.$$

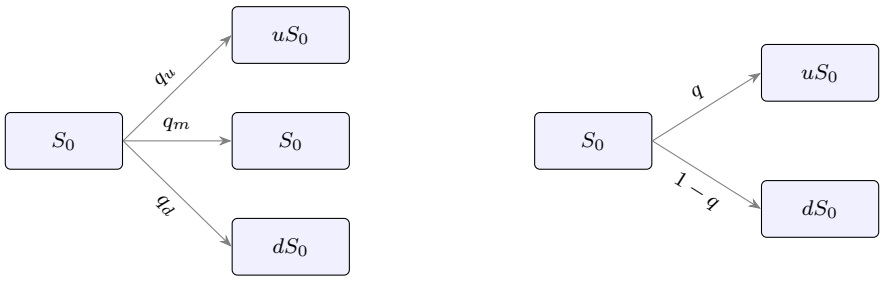


Figure 3: Une période dans l'arbre trinomial (à gauche) et dans l'arbre binomial (à droite).

3.2 Un marché incomplet

La réplication n'est plus possible

Reprenons la démarche du chapitre 2 avec trois états. Le portefeuille (Δ, B) doit reproduire le payoff dans *chacun* des trois états :

$$\begin{cases} \Delta\, uS_0 + RB = h(uS_0) \\ \Delta\, S_0 \;\; + RB = h(S_0) \\ \Delta\, dS_0 + RB = h(dS_0) \end{cases}$$

On a maintenant **trois équations pour deux inconnues**. En général ce système n'a pas de solution : une option quelconque n'est pas duplicable avec le seul sous-jacent et l'actif sans risque. Le marché trinomial est dit **incomplet**.

Exemple. Avec les données du cours ($S_0 = K = 100$, $u = 1{,}1$, $m = 1$, $d = 0{,}9$, $R = 1{,}05$; sur une seule période, la condition $d = 1/u$ n'est pas nécessaire), le call a pour payoffs 10, 0 et 0. Les deux dernières équations donnent $100\Delta + 1{,}05B = 0$ et $90\Delta + 1{,}05B = 0$, donc $\Delta = 0$ et $B = 0$, ce qui contredit la première ($0 = 10$). Aucun portefeuille ne réplique ce call.

Une infinité de probabilités risque-neutres

Le cours montre que l'absence d'arbitrage équivaut à l'existence d'une probabilité risque-neutre, c'est-à-dire d'une probabilité (q_u, q_m, q_d) chargeant les trois états et rendant le prix actualisé martingale :

$$q_u + q_m + q_d = 1, \qquad q_u\, u + q_m\, m + q_d\, d = R, \qquad q_u, q_m, q_d > 0.$$

C'est un système de **deux équations à trois inconnues** : il admet en général une *infinité* de solutions. Ici l'absence d'arbitrage s'écrit $d < R < u$, comme dans le binomial (l'actif sans risque doit rapporter plus que le pire état et moins que le meilleur).

Sur l'exemple, posons $q_d = t$. Le système donne $0{,}1\,q_u - 0{,}1\,t = 0{,}05$, soit

$$q_u = 0{,}5 + t, \qquad q_m = 0{,}5 - 2t, \qquad q_d = t, \qquad 0 < t < 0{,}25.$$

Chaque valeur de t définit une probabilité risque-neutre différente, et donc un prix différent pour le call :

$$C(t) = \frac{10\,(0{,}5 + t)}{1{,}05} \in \ ]\,4{,}76\ ;\ 7{,}14\,[.$$

Ce que change l'incomplétude

- L'absence d'arbitrage ne fixe plus **un** prix, mais un **intervalle de prix** : ici, tout prix du call entre 4,76 et 7,14 est sans arbitrage.
- La borne haute 7,14 est le **prix de sur-réplication** : le coût du moins cher des portefeuilles qui rapportent *au moins* le payoff dans tous les états. On retrouve exactement le prix binomial du chapitre 2.
- Le choix d'un prix dans l'intervalle revient à choisir une probabilité risque-neutre : c'est un **choix de modèle**, qui se fait par *calibrage* (section 3.3).
- La parité call–put reste vraie pour *toutes* les probabilités risque-neutres : le put vaut $P(t) = \frac{10\,t}{1{,}05}$ et $C(t) - P(t) = \frac{5}{1{,}05} = 100 - \frac{100}{1{,}05}$. En effet, le contrat « call moins put » a le payoff $S_T - K$, qui, lui, est réplicable.

3.3 Calibrage de l'arbre trinomial sur la volatilité

Pour lever l'indétermination, on choisit les paramètres de façon que l'arbre reproduise les deux premiers moments du rendement logarithmique du modèle de Black–Scholes, comme dans le calibrage CRR. Suivant Boyle [5] et Kamrad–Ritchken [6], on introduit un **paramètre d'écartement** $\lambda \geq 1$:

$$u = e^{\lambda\sigma\sqrt{\Delta t}}, \qquad m = 1, \qquad d = \frac{1}{u}, \qquad R = e^{r\Delta t}.$$

On *fixe* ensuite la probabilité de l'état stable, puis on impose la condition martingale, comme pour q dans le binomial :

$$\boxed{q_m = 1 - \frac{1}{\lambda^2}, \qquad q_u = \frac{R - q_m - (1 - q_m)\,d}{u - d}, \qquad q_d = 1 - q_m - q_u.}$$

Par construction, $\mathbb{E}_{\mathbb{Q}}[S_{k+1} \mid S_k] = R\,S_k$ *exactement*, si bien que la parité call–put est vérifiée exactement dans l'arbre. Un développement limité donne, en notant $\nu = r - \sigma^2/2$,

$$q_u \approx \frac{1}{2\lambda^2} + \frac{\nu\sqrt{\Delta t}}{2\lambda\sigma}, \qquad q_d \approx \frac{1}{2\lambda^2} - \frac{\nu\sqrt{\Delta t}}{2\lambda\sigma},$$

qui sont les probabilités de Kamrad et Ritchken. On vérifie alors que le rendement logarithmique sur un pas, qui vaut $+\lambda\sigma\sqrt{\Delta t}$, 0 ou $-\lambda\sigma\sqrt{\Delta t}$, a pour moyenne $(q_u - q_d)\lambda\sigma\sqrt{\Delta t} \approx \nu\,\Delta t$ et pour variance $\approx (q_u + q_d)\lambda^2\sigma^2\Delta t = \sigma^2\Delta t$: l'arbre imite le mouvement brownien géométrique, quel que soit λ.

Choix de λ

- $\lambda = 1$ donne $q_m = 0$: l'état stable disparaît et l'on retrouve **exactement l'arbre binomial CRR**. Le trinomial *généralise* donc le binomial.
- $\lambda = \sqrt{3}$ donne $q_m = 2/3$: c'est le choix présenté par Hull [3], avec $q_u \approx q_d \approx 1/6$.
- $\lambda = \sqrt{3/2} \approx 1{,}22$ donne $q_m = 1/3$: les trois états ont des probabilités proches. C'est la valeur mise en avant par Kamrad et Ritchken [6] pour sa bonne convergence ; nos tests du chapitre 6 le confirment.
- Il faut $\lambda \geq 1$ pour que $q_m \geq 0$, et Δt assez petit pour que $q_d > 0$: à défaut, le code signale une erreur.

3.4 Lien exact avec le binomial : deux pas binomiaux font un pas trinomial

Considérons deux pas successifs d'un arbre CRR de durée h. Les prix possibles sont $u_b^2 S$, S et $d_b^2 S$, avec les probabilités q^2, $2q(1-q)$ et $(1-q)^2$. C'est un arbre trinomial de pas $\Delta t = 2h$, avec $u = u_b^2 = e^{2\sigma\sqrt{h}} = e^{\sqrt{2}\,\sigma\sqrt{\Delta t}}$, c'est-à-dire $\lambda = \sqrt{2}$, et $q_m = 2q(1-q) \approx \frac{1}{2} = 1 - \frac{1}{\lambda^2}$. Le binomial à $2N$ pas est donc (au premier ordre) un trinomial particulier à N pas : les deux modèles appartiennent à la même famille.

3.5 Valorisation par récurrence rétrograde

La récurrence rétrograde se généralise directement, avec une branche de plus :

$$V_N(S) = h(S), \qquad V_k(S) = \frac{1}{R}\Big[q_u\, V_{k+1}(uS) + q_m\, V_{k+1}(S) + q_d\, V_{k+1}(dS)\Big].$$

Pour une option européenne, elle équivaut à $V_0 = \mathbb{E}_{\mathbb{Q}}[h(S_N)]/R^N$, où le nombre de hausses moins le nombre de baisses suit une loi trinomiale.

Exemple chiffré à deux périodes

Prenons $S_0 = K = 100$, $u = 1{,}25$, $m = 1$, $d = 0{,}8$ (on a bien $ud = 1$), $R = 1{,}05$, et choisissons $q_m = \frac{1}{2}$. La condition martingale s'écrit $1{,}25\,q_u + 0{,}5 + 0{,}8\,q_d = 1{,}05$ avec $q_u + q_d = \frac{1}{2}$, d'où

$$q_u = \tfrac{1}{3}, \qquad q_m = \tfrac{1}{2}, \qquad q_d = \tfrac{1}{6}.$$

La figure 4 donne l'arbre d'un call européen. Par exemple, au nœud $S = 125$ du pas 1, $V = \frac{\frac{1}{3}\times 56{,}25 + \frac{1}{2}\times 25 + \frac{1}{6}\times 0}{1{,}05} \approx 29{,}76$, et le prix en $t = 0$ vaut $\frac{\frac{1}{3}\times 29{,}76 + \frac{1}{2}\times 7{,}94 + \frac{1}{6}\times 0}{1{,}05} \approx \mathbf{13{,}23}$.

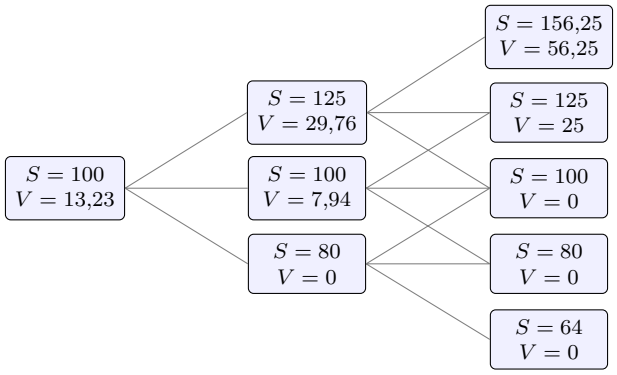


Figure 4: Call européen de strike 100 dans un arbre trinomial à 2 périodes ($q_u = \frac{1}{3}$, $q_m = \frac{1}{2}$, $q_d = \frac{1}{6}$).

Le même arbre donne un put européen de **3**,**93**, et l'on vérifie la parité : $13{,}23 - 3{,}93 = 9{,}30 = 100 - \frac{100}{1{,}05^2}$.

3.6 Couverture et Greeks dans l'arbre trinomial

Puisque le marché est incomplet, il n'existe plus de couverture parfaite : avec un seul actif risqué, on ne peut pas neutraliser trois scénarios à la fois. On utilise en pratique le delta « central »

$$\Delta_{k,j} = \frac{V_{k+1}(uS) - V_{k+1}(dS)}{(u-d)\,S},$$

qui couvre les deux scénarios extrêmes ; il reste un risque résiduel dans l'état stable, qui disparaît quand $\Delta t \to 0$. Sur l'exemple, $\Delta_0 = \frac{29{,}76-0}{125-80} \approx 0{,}66$.

L'arbre trinomial a en revanche un avantage pratique pour les sensibilités : dès le premier pas, on dispose de *trois* nœuds dS_0, S_0 et uS_0, ce qui permet de lire le delta *et le gamma* directement en $t = 0$:

$$\Gamma_0 \approx \frac{\dfrac{V_1(uS_0) - V_1(S_0)}{uS_0 - S_0} - \dfrac{V_1(S_0) - V_1(dS_0)}{S_0 - dS_0}}{\frac{1}{2}\,(uS_0 - dS_0)},$$

alors que dans le binomial il faut attendre le pas 2.

3.7 Interprétation : un schéma aux différences finies

En variable $x = \ln S$, l'arbre trinomial est une grille régulière de pas $\Delta x = \lambda\sigma\sqrt{\Delta t}$, et la récurrence rétrograde relie chaque nœud à ses trois voisins du pas suivant. C'est exactement un **schéma aux différences finies explicite** pour l'équation aux dérivées partielles de Black–Scholes (Brennan et Schwartz [7]). La condition $q_m \geq 0$, c'est-à-dire $\lambda \geq 1$, s'écrit $\sigma^2 \Delta t \leq \Delta x^2$: c'est la condition de stabilité classique du schéma explicite. Le trinomial fait ainsi le pont entre les arbres vus en cours et les méthodes d'EDP.

3.8 Synthèse : binomial ou trinomial ?

	Binomial (CRR)	**Trinomial**
États par pas	2 (hausse, baisse)	3 (hausse, stable, baisse)
Nœuds au pas k	$k+1$	$2k+1$
Marché	complet	incomplet
Probabilité risque-neutre	unique, $q = \frac{R-d}{u-d}$	non unique, fixée par calibrage
Réplication	parfaite	impossible en général
Paramètres	$u = e^{\sigma\sqrt{\Delta t}}$	$u = e^{\lambda\sigma\sqrt{\Delta t}}$, $\lambda \geq 1$
Gamma en $t = 0$	non (pas 2)	oui (pas 1)
Coût par pas	$\approx N$ opérations	$\approx 2N$ opérations

Table 1: Comparaison théorique des deux modèles.

Chapitre 4: Convergence vers Black–Scholes

Ce chapitre répond à la dernière partie de notre problématique : *les prix donnés par les arbres convergent-ils vers celui de Black–Scholes lorsque le nombre de périodes N augmente, et à quelle vitesse ?* Black–Scholes joue ici le rôle de référence : c'est le prix exact d'une option européenne dans le modèle en temps continu.

4.1 Rappel : la formule de Black–Scholes

Pour une option européenne sans dividende, avec un taux continu r et une volatilité σ constants,

$$C = S_0\,\mathcal{N}(d_1) - Ke^{-rT}\mathcal{N}(d_2), \qquad P = Ke^{-rT}\mathcal{N}(-d_2) - S_0\,\mathcal{N}(-d_1),$$

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T},$$

où $\mathcal{N}$ est la fonction de répartition de la loi normale centrée réduite.

4.2 Énoncé du résultat

On fixe S_0, K, T, r et σ, puis on fait croître N (donc $\Delta t = T/N \to 0$), avec le calibrage CRR (section 2.6) pour le binomial et le calibrage de la section 3.3 pour le trinomial. On note $V_0^{(N)}$ le prix donné par l'arbre et V_0^{BS} le prix de Black–Scholes.

Proposition 1 (Convergence vers Black–Scholes) *Pour un call ou un put européen sans dividende, dans l'arbre binomial CRR comme dans l'arbre trinomial (pour tout $\lambda \geq 1$ fixé),* $\lim_{N\to\infty} V_0^{(N)} = V_0^{\mathrm{BS}}$.

La réponse à notre question est donc **oui**, pour les deux modèles. Ce résultat est dû à Cox, Ross et Rubinstein [4] pour le binomial et à Boyle [5] et Kamrad–Ritchken [6] pour le trinomial ; nous en donnons l'idée ci-dessous, sans la démonstration complète.

4.3 Pourquoi le prix converge

Dans les deux arbres, $\ln S_N = \ln S_0 + \sum_{k=1}^{N} X_k$, où les X_k sont des rendements logarithmiques indépendants et de même loi sous la probabilité risque-neutre :

- binomial : $X_k = \pm\sigma\sqrt{\Delta t}$;
- trinomial : $X_k \in \{+\lambda\sigma\sqrt{\Delta t},\ 0,\ -\lambda\sigma\sqrt{\Delta t}\}$.

Les deux calibrages ont été construits pour que chaque pas ait une moyenne $\approx (r - \sigma^2/2)\Delta t$ et une variance $\approx \sigma^2\Delta t$. En sommant N pas, la moyenne vaut $(r - \sigma^2/2)T$ et la variance $\sigma^2 T$. Par le théorème central limite, $\ln S_N$ converge en loi vers une loi normale de paramètres $\big(\ln S_0 + (r - \sigma^2/2)T,\ \sigma^2 T\big)$, c'est-à-dire que S_N converge vers la loi log-normale du modèle de Black–Scholes sous la probabilité risque-neutre. Le prix, qui est une espérance actualisée du payoff, converge donc vers le prix de Black–Scholes.

On remarque que, *dans la limite*, l'incomplétude du trinomial disparaît : quel que soit le choix de λ (donc de la probabilité risque-neutre), le prix converge vers le même prix de Black–Scholes, dans un marché continu qui, lui, est complet (proposition 6.5.3 du cours [1]).

4.4 Vitesse de convergence et oscillations

Pour les deux arbres, l'erreur est d'ordre $1/N$. Elle n'est cependant pas régulière :

- dans le **binomial**, les nœuds finaux sont $S_0 u^{2j-N}$: leur position change selon que N est pair ou impair. Le strike tombe donc alternativement plus ou moins près d'un nœud, et le prix *oscille* autour de la limite d'un pas à l'autre (effet « pair/impair ») ;
- dans le **trinomial**, les nœuds finaux $S_0 u^j$, $j = -N, \dots, N$, couvrent *tous* les entiers : il n'y a plus d'effet de parité. L'erreur varie lentement avec N, sans changer de signe à chaque pas.

Ces deux comportements, ainsi que l'effet de λ, sont vérifiés numériquement au chapitre 6.

Chapitre 5: Implémentation

5.1 Choix techniques et architecture

Le code est écrit en Python : la logique de pricing (arbres, probabilités risque-neutres, récurrence rétrograde, delta, gamma) suit directement les formules des chapitres 2 et 3. La bibliothèque `numpy` sert au calcul vectoriel, et `scipy.stats.norm` à la fonction de répartition normale dans Black–Scholes, qui sert de *référence de validation*.

Le projet comporte cinq parties : le modèle binomial (`binomial.py`), le modèle trinomial (`trinomial.py`), le script `figures_trinomial.py`, qui produit les figures du chapitre 6, un module de données de marché et de volatilité implicite, et une interface web interactive. Les deux modèles sont joints en annexe.

5.2 Algorithmes

```
BINOMIAL                                  TRINOMIAL
1. q = (R-d)/(u-d)  (vérifier d<R<u)      1. qm = 1-1/lambda^2 ; qu, qd (martingale)
2. V[j] = h(S0 u^j d^(N-j)), j=0..N       2. V[j] = h(S0 u^j),  j=-N..N
3. pour k = N-1 .. 0 :                    3. pour k = N-1 .. 0 :
     V[j] = (q V[j+1] + (1-q) V[j]) / R         V[j] = (qu V[j+1] + qm V[j] + qd V[j-1]) / R
4. retourner V[0]                         4. retourner V[0]
```

Le binomial traite environ $N^2/2$ nœuds, le trinomial environ N^2 : pour un même N, le trinomial coûte environ deux fois plus cher. La mémoire est en $O(N)$ dans les deux cas pour les versions vectorielles.

5.3 Le cœur du code

Binomial

```python
def prix_binomial(S0, K, N, u, d, R, option="call"):
    q = proba_risque_neutre(u, d, R)
    j = np.arange(N + 1)
    V = payoff(S0 * u**j * d**(N - j), K, option)       # maturité
    for k in range(N - 1, -1, -1):
        V = (q * V[1:k + 2] + (1 - q) * V[0:k + 1]) / R   # continuation
    return float(V[0])
```

Trinomial

```python
def probas_risque_neutre(u, d, R, pm):
    pu = (R - pm - (1 - pm) * d) / (u - d)      # condition martingale
    pd = 1.0 - pm - pu
    if not (0 < pu < 1 and 0 < pd < 1 and 0 <= pm < 1):
        raise ValueError("Probabilités hors de ]0,1[")
    return pu, pm, pd

def prix_trinomial(S0, K, N, u, d, R, pm, option="call"):
    pu, pm, pd = probas_risque_neutre(u, d, R, pm)
    j = np.arange(-N, N + 1)
    V = payoff(S0 * u**j, K, option)                          # maturité
    for k in range(N - 1, -1, -1):
        V = (pu * V[2:] + pm * V[1:-1] + pd * V[:-2]) / R      # continuation
    return float(V[0])
```

Dans la boucle trinomiale, `V[2:]`, `V[1:-1]` et `V[:-2]` contiennent respectivement les valeurs après une hausse, une stabilité et une baisse : chaque passage réduit le vecteur de deux cases, puisque le pas k compte $2k+1$ nœuds. La fonction `arbre_trinomial` stocke en plus tous les prix et valeurs, ce qui permet de lire le delta et le gamma au premier pas.

5.4 Greeks et volatilité implicite

Dans le binomial, le delta, le gamma et le theta sont lus dans l'arbre à partir des nœuds des pas 1 et 2 (méthode classique présentée par Hull) ; le vega et le rho sont obtenus par différences finies centrées. Dans le trinomial, le delta et le gamma sont lus directement sur les trois nœuds du pas 1. La *volatilité implicite* d'une option cotée est la valeur de σ pour laquelle le prix du modèle égale le prix de marché ; elle est calculée par dichotomie, le prix du modèle étant croissant en σ.

5.5 Validation par tests automatiques

1. **Exemple du cours** (binomial) : $q = 0{,}75$, call $= 7{,}1429$, put $= 2{,}3810$, $\Delta = 0{,}5$.
2. **Exemple du rapport** (trinomial) : $u = 1{,}25$, $d = 0{,}8$, $R = 1{,}05$, $q_m = \frac{1}{2}$ donnent $q_u = \frac{1}{3}$ et $q_d = \frac{1}{6}$.
3. **Lien entre les modèles** : avec $\lambda = 1$, le trinomial redonne le binomial CRR à 10^{-10} près (call et put).
4. **Parité call–put** exacte dans les deux arbres (précision 10^{-9}).
5. **Cohérence** des implémentations vectorielles et « arbre complet ».
6. **Convergence** vers Black–Scholes (écart $< 5 \cdot 10^{-3}$ pour $N = 2000$ en binomial, $N = 1000$ en trinomial).

Tous les tests passent.

5.6 Interface interactive

L'interface est une page web qui exécute l'algorithme binomial dans le navigateur. Elle permet de modifier S_0, K, T, r, σ, N, de choisir un call ou un put, et affiche le prix binomial, le prix de Black–Scholes, le delta et le gamma, l'arbre animé (le survol d'un nœud montre son portefeuille de réplication), la convergence vers Black–Scholes et le prix en fonction de S_0. Une section « marché réel » compare le modèle à des prix cotés. Les comparaisons avec le trinomial sont produites par le script Python. Les captures d'écran de l'interface sont regroupées en **annexe A**. L'interface propose aussi un mode « option américaine » (exercice possible avant l'échéance), qui permet de voir la différence de prix avec l'européenne ; ce cas, qui relève de la théorie de l'arrêt optimal, sort du cadre de ce rapport (voir chapitre 7).

Chapitre 6: Résultats et comparaison des deux modèles

Cas de référence : $S_0 = 100$, $K = 100$, $T = 1$ an, $r = 5\,\%$, $\sigma = 20\,\%$, sans dividendes. Black–Scholes donne $C = 10{,}4506$ et $P = 5{,}5735$.

6.1 Convergence du binomial vers Black–Scholes

N	Call européen	Erreur vs BS	Put européen
10	10,2534	0,1972	5,3764
50	10,4107	0,0399	5,5336
100	10,4306	0,0200	5,5536
500	10,4466	0,0040	5,5695
1000	10,4486	0,0020	5,5715
2000	10,4496	0,0010	5,5725
Black–Scholes	10,4506	–	5,5735

Table 2: Prix binomiaux (CRR) selon le nombre de pas N.

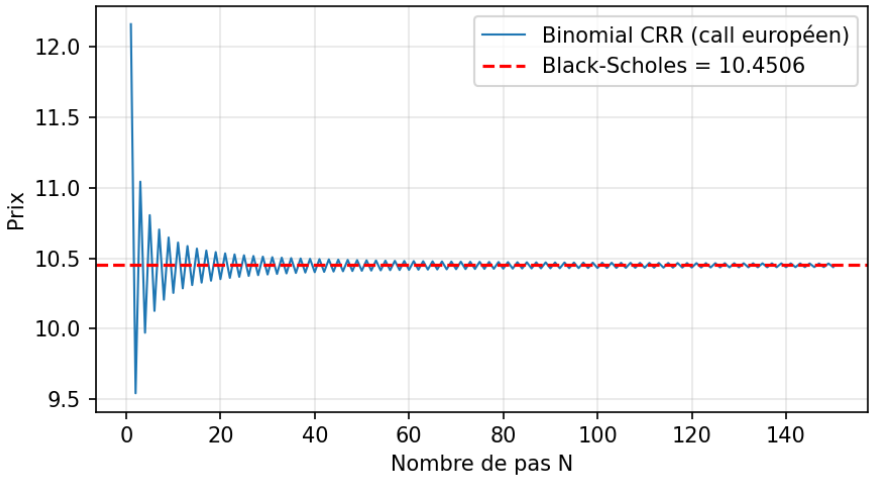


Figure 5: Convergence du call européen binomial vers Black–Scholes : le prix oscille autour de la valeur limite.

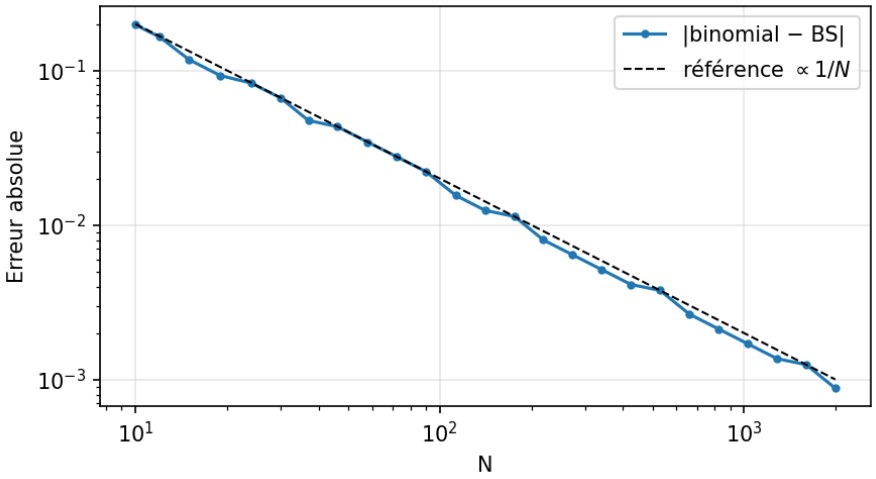


Figure 6: Erreur absolue du binomial en échelle log–log : pente -1, soit une erreur en $O(1/N)$.

Lecture. L'erreur est divisée par 10 quand N est multiplié par 10 : la convergence est d'ordre 1, conformément au chapitre 4. Les oscillations viennent de la position du strike par rapport aux nœuds finaux.

6.2 Convergence du trinomial et comparaison

N	Call européen	Erreur vs BS	Put européen
10	10,3941	0,0565	5,5170
50	10,4390	0,0116	5,5619
100	10,4448	0,0058	5,5677
500	10,4494	0,0012	5,5724
1000	10,4500	0,0006	5,5729
2000	10,4503	0,0003	5,5732
Black–Scholes	10,4506	–	5,5735

Table 3: Prix trinomiaux ($\lambda = \sqrt{3/2}$) selon le nombre de pas N.

À nombre de pas égal, l'erreur du trinomial ($\lambda = \sqrt{3/2}$) sur le call à la monnaie est environ **3,4 fois plus faible** que celle du binomial : 0,0058 contre 0,0200 pour $N = 100$. L'erreur reste d'ordre $1/N$: elle est divisée par 10 quand N est multiplié par 10.

Le cas à la monnaie est particulier (le strike tombe exactement sur un nœud). Pour observer les oscillations, la figure 7 prend un call *hors de la monnaie* ($K = 110$).

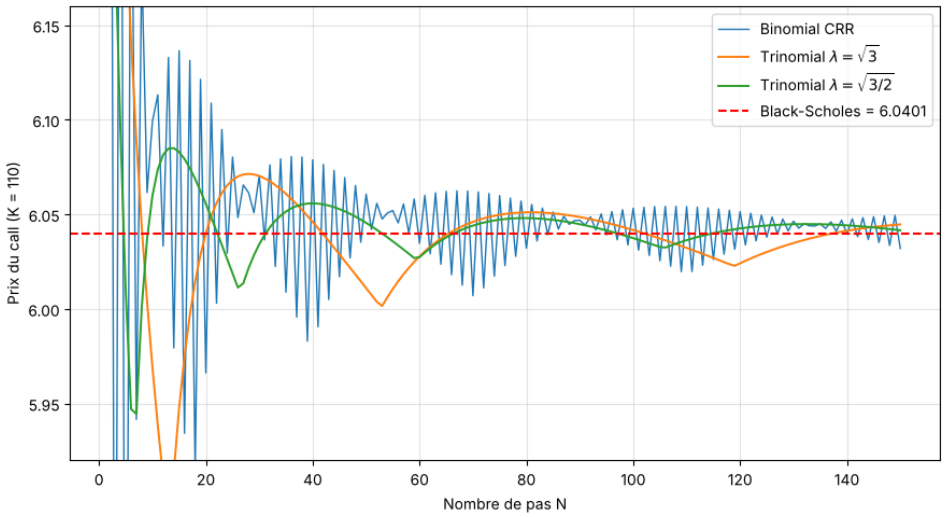


Figure 7: Call européen $K = 110$: le binomial oscille en dents de scie d'un pas à l'autre, le trinomial varie lentement autour de la limite.

Lecture. Le binomial change de côté à chaque pas (effet pair/impair). Le trinomial n'a plus cet effet : l'erreur varie lentement, en « vagues » dont l'amplitude décroît. Pour $\lambda = \sqrt{3/2}$, l'amplitude est nettement plus faible ; pour $\lambda = \sqrt{3}$, les vagues sont plus larges, car les nœuds sont plus espacés.

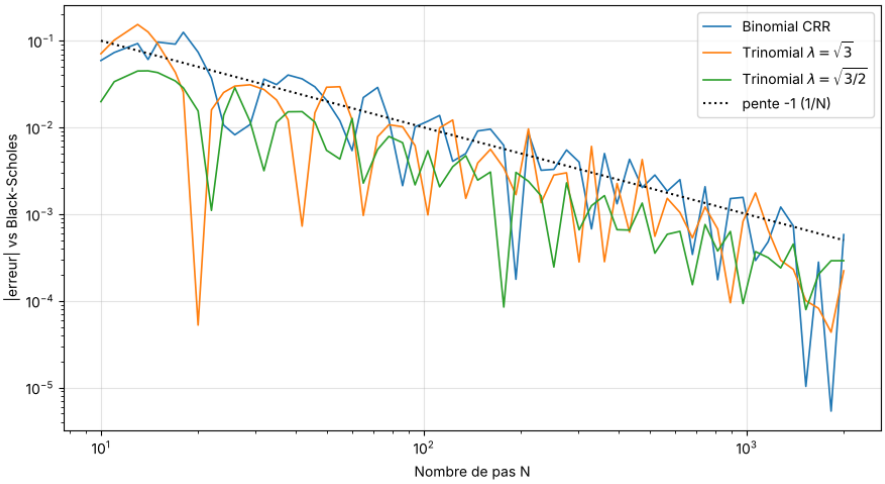


Figure 8: Erreur absolue ($K = 110$) en échelle log–log : les trois arbres suivent la pente -1.

6.3 Rôle du paramètre λ

La figure 9 mesure l'erreur moyenne du call européen sur trois strikes (90, 100, 110) et sur N allant de 100 à 200, en fonction de λ.

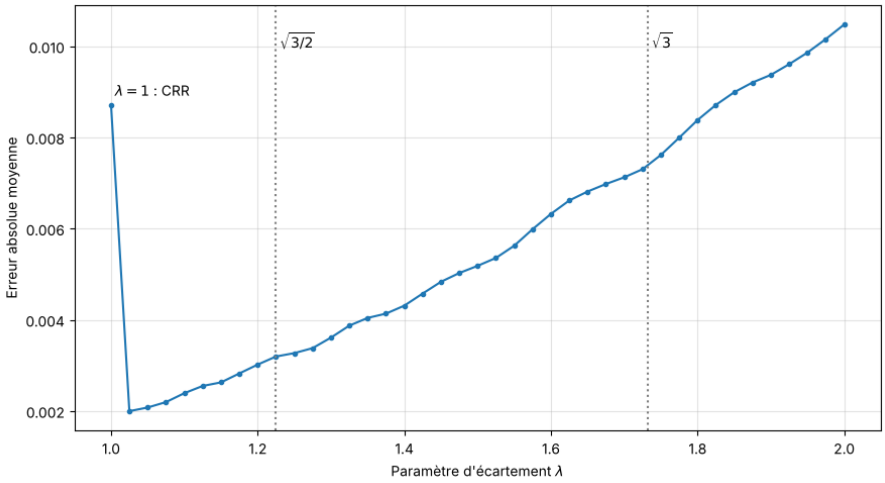


Figure 9: Erreur absolue moyenne du trinomial en fonction de λ ($\lambda = 1$: binomial CRR).

Lecture. Dès que λ dépasse 1, l'erreur moyenne chute brutalement : l'état stable apparaît et l'effet pair/impair disparaît. Elle remonte ensuite quand λ augmente, car les nœuds s'écartent ($\Delta x = \lambda\sigma\sqrt{\Delta t}$) et la grille devient plus grossière. Le choix de Hull ($\lambda = \sqrt{3}$) fait à peine mieux que le binomial ; $\lambda = \sqrt{3/2}$ divise l'erreur moyenne par près de trois.

6.4 Précision à temps de calcul égal

Le trinomial coûte environ deux fois plus cher par pas. La comparaison honnête se fait donc à *temps de calcul égal*.

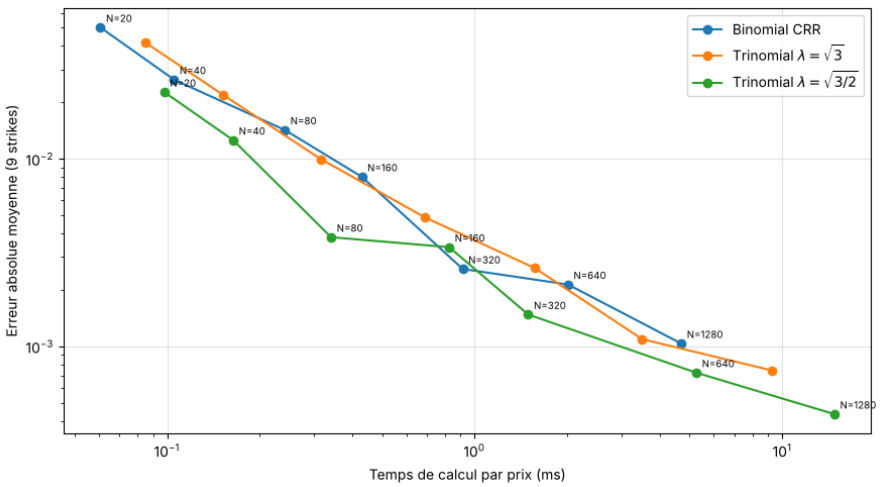


Figure 10: Erreur moyenne (9 strikes de 90 à 110) en fonction du temps de calcul par prix.

Lecture. À temps de calcul égal, le trinomial avec $\lambda = \sqrt{3/2}$ reste plus précis que le binomial, mais le gain est plus modeste et plus irrégulier qu'à N égal : selon la zone, l'erreur est de 1 à 3 fois plus faible, soit un facteur d'environ 1,4 en moyenne. Avec $\lambda = \sqrt{3}$, il n'y a pas de gain. Le principal avantage du trinomial est donc la **régularité** de la convergence, plus que la vitesse.

6.5 Sensibilité aux paramètres

		Call européen			Put européen		
Paramètre	Valeur	Binomial	Trinomial	BS	Binomial	Trinomial	BS
σ	10%	6,80	6,80	6,80	1,93	1,93	1,93
	20%	10,45	10,45	10,45	5,57	5,57	5,57
	30%	14,23	14,23	14,23	9,35	9,35	9,35
	40%	18,02	18,02	18,02	13,14	13,14	13,15
r	1%	8,43	8,43	8,43	7,43	7,44	7,44
	5%	10,45	10,45	10,45	5,57	5,57	5,57
	10%	13,27	13,27	13,27	3,75	3,75	3,75
K	90	16,70	16,70	16,70	2,31	2,31	2,31
	100	10,45	10,45	10,45	5,57	5,57	5,57
	110	6,04	6,04	6,04	10,68	10,68	10,68
T (ans)	0,5	6,89	6,89	6,89	4,42	4,42	4,42
	1	10,45	10,45	10,45	5,57	5,57	5,57
	2	16,12	16,12	16,13	6,60	6,61	6,61

Table 4: Sensibilités ($N = 500$, trinomial avec $\lambda = \sqrt{3/2}$, autres paramètres au cas de référence).

Les deux arbres donnent les mêmes prix que Black–Scholes au centime près (à $\pm 0{,}01$ près dans trois cas). Les variations suivent l'intuition financière :

- $\sigma \nearrow$: le call et le put augmentent, car le détenteur profite des grands mouvements sans subir les pertes au-delà de la prime.
- $r \nearrow$: le call monte et le put baisse, car le strike payé (ou reçu) à l'échéance vaut moins aujourd'hui.
- $K \nearrow$: le call baisse et le put monte.
- $T \nearrow$: les deux options prennent de la valeur temps.

6.6 Delta, gamma et couverture

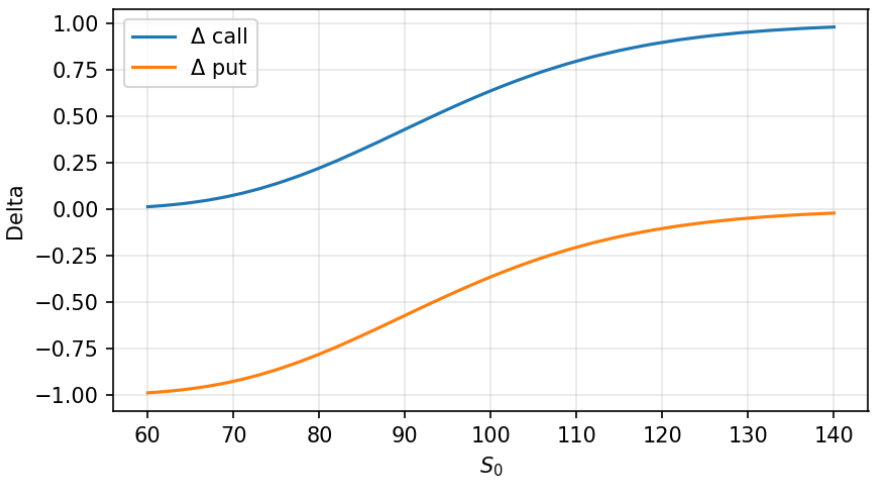


Figure 11: Delta à $t = 0$ du call et du put européens en fonction de S_0.

	Binomial ($N = 500$)	Trinomial ($N = 500$, $\lambda = \sqrt{3/2}$)	Black–Scholes
Δ call	0,6368	0,6367	0,6368
Δ put	$-0{,}3632$	$-0{,}3633$	$-0{,}3632$
Γ	0,0188	0,0188	0,0188

Table 5: Delta et gamma au cas de référence.

Les deux arbres retrouvent le delta de Black–Scholes, $\Delta_C = \mathcal{N}(d_1) = \mathcal{N}(0{,}35) \approx 0{,}6368$, et vérifient $\Delta_C - \Delta_P = 1$ (dérivée de la parité). Le gamma trinomial est lu dès le premier pas et vaut 0,01878, pour une valeur exacte de $\frac{\mathcal{N}'(d_1)}{S_0\sigma\sqrt{T}} = 0{,}01876$.

6.7 Confrontation au marché réel

L'interface récupère, pour un titre coté, son prix, sa volatilité historique sur 30 jours et la chaîne d'options d'une échéance (données Yahoo Finance, indicatives). Pour chaque strike, elle compare le prix de marché (moyenne entre l'offre et la demande) au prix du modèle binomial calculé avec σ égale à la volatilité historique, et calcule la volatilité implicite du modèle, comparée à celle publiée par Yahoo. Les options cotées sur actions américaines pouvant être exercées avant l'échéance, l'interface utilise pour cette comparaison la variante « américaine » de l'arbre ; pour des calls sur un titre qui verse peu de dividendes, elle donne pratiquement le même prix que l'européenne. Comme les prix binomiaux et trinomiaux coïncident à quelques millièmes près pour N de l'ordre de quelques centaines, cette comparaison vaut pour les deux modèles.

Sur les calls Apple (AAPL) d'échéance 14 octobre 2026 (figure 14, annexe A), avec un cours de 341,23 et une volatilité historique de 21,3 %, l'écart entre le prix du modèle et le prix de marché ne dépasse pas 0,65 sur les strikes affichés. Les volatilités implicites très élevées pour les strikes bas ne sont pas significatives : à deux jours de l'échéance, ces calls très dans la monnaie valent presque leur valeur intrinsèque, et une petite différence de prix se traduit par un grand écart de volatilité.

N.B :
(i) *La volatilité* : le modèle utilise une volatilité *historique* (passée), alors que le marché cote une volatilité *implicite* (anticipée) ; c'est probablement la première source d'écart.
(ii) *Les dividendes* : le modèle n'en tient pas compte ; pour un titre qui en verse, les calls sont surévalués et les puts sous-évalués, avec un effet qui dépend du dividende et de l'échéance.
(iii) *Le taux sans risque* : il doit être cohérent avec la devise et l'échéance (taux de bons du Trésor américain pour un titre américain) ; ici $r = 5\%$.
(iv) *L'écart offre–demande* : le prix de marché est une moyenne, pas un prix d'exécution.

Chapitre 7: Limites et extensions

7.1 Limites

Les deux modèles supposent une volatilité et un taux constants, aucun dividende et aucun frais de transaction. Leur convergence est d'ordre 1 : pour gagner une décimale, il faut multiplier N par 10, donc le temps de calcul par 100. Le binomial oscille d'un pas à l'autre ; le trinomial supprime cet effet mais repose sur un choix de calibrage (λ), conséquence directe de l'incomplétude du marché à trois états : son prix n'est pas imposé par l'arbitrage seul, et sa couverture n'est pas parfaite à pas de temps fixé.

7.2 Extensions

- **Dividendes** : avec un rendement de dividende continu δ, on remplace $R = e^{r\Delta t}$ par $e^{(r-\delta)\Delta t}$ dans la condition martingale, dans les deux arbres.
- **Options exotiques** : les arbres se prêtent à des adaptations (options bermudéennes, digitales, à barrière). Le trinomial est ici particulièrement utile : grâce à son paramètre λ, on peut placer une *rangée de nœuds exactement sur la barrière*, ce qui supprime une grande partie de l'erreur.
- **Arbres trinomiaux de taux** : le modèle de Hull–White est implémenté sur un arbre trinomial, car l'état stable permet de représenter le retour à la moyenne des taux.
- **Paramétrisations plus fines** (Jarrow–Rudd, Leisen–Reimer) pour une convergence plus lisse du binomial, et **extrapolation de Richardson** sur deux valeurs de N pour gagner un ordre de convergence.
- **Différences finies** : le trinomial étant un schéma explicite, l'étape suivante naturelle est le schéma implicite ou de Crank–Nicolson, sans contrainte de stabilité.
- **Options américaines** : une option américaine peut être exercée à toute date avant l'échéance. Dans un arbre, il suffit de comparer en chaque nœud la valeur de continuation au gain d'exercice immédiat et d'en prendre le maximum ; la justification théorique repose sur l'enveloppe de Snell et les temps d'arrêt (chapitre 4 du cours). Notre interface propose déjà ce mode. On sait notamment qu'un call américain sans dividende ne s'exerce jamais avant l'échéance (Merton [9]), alors qu'un put américain vaut plus que le put européen. La méthode de Longstaff et Schwartz [11] permet aussi de les traiter par Monte-Carlo.

Conclusion générale

Le modèle binomial se construit à partir de deux idées : la réplication et l'absence d'arbitrage, qui donnent la probabilité risque-neutre q et la récurrence rétrograde. Implémenté en Python, il reproduit l'exemple du cours, vérifie la parité call–put et converge vers Black–Scholes avec une erreur en $1/N$, mais de façon oscillante.

Le modèle trinomial ajoute un état stable. Ce simple ajout change la nature du marché : avec trois états et deux actifs, le marché devient incomplet, la réplication parfaite disparaît et la probabilité risque-neutre n'est plus unique. L'absence d'arbitrage ne donne plus qu'un intervalle de prix, dont la borne haute est, sur l'exemple du cours, le prix binomial. On lève l'indétermination en calibrant l'arbre sur la volatilité, grâce au paramètre λ. Pour $\lambda = 1$, on retrouve exactement le binomial ; pour $\lambda > 1$, l'effet pair/impair disparaît. Avec $\lambda = \sqrt{3/2}$, l'erreur est environ trois fois plus faible qu'en binomial pour le même nombre de pas, et reste plus faible à temps de calcul égal.

Les deux arbres retrouvent aussi les sensibilités de Black–Scholes : le delta et le gamma lus dans l'arbre coïncident avec les valeurs exactes, et le trinomial donne le gamma dès le premier pas.

En réponse à notre problématique : les deux modèles valorisent correctement les options européennes et convergent tous deux vers Black–Scholes. Le binomial reste le meilleur outil pour *comprendre* la valorisation, puisque tout s'y explique par la réplication. Le trinomial, bien paramétré, est le meilleur outil pour *calculer*, au prix d'une hypothèse de calibrage. L'interface permet enfin d'explorer ces résultats et de les confronter au marché, en gardant à l'esprit les limites communes aux deux modèles.

Annexe A : Captures de l'interface

Les captures ci-dessous montrent l'interface web décrite à la section « Interface interactive » du chapitre 5, avec les paramètres du cas de référence ($S_0 = K = 100$, $T = 1$ an, $r = 5\,\%$, $\sigma = 20\,\%$, $N = 100$ pas). Le mode « exercice américain » n'est pas activé : seules les options européennes sont étudiées dans ce rapport.

En haut de l'écran, l'interface affiche le prix binomial, le prix de Black–Scholes, le delta et la probabilité risque-neutre q. On retrouve exactement les valeurs du tableau de convergence du chapitre 6 pour $N = 100$: 10,4306 pour le call et 5,5536 pour le put. L'arbre n'est dessiné que sur ses quatre premiers pas, pour rester lisible : les valeurs affichées sur ses nœuds correspondent donc à un arbre à 4 pas (prix 9,97 pour le call, 5,09 pour le put). En dessous, l'interface trace la convergence du prix binomial vers Black–Scholes et le prix de l'option en fonction de S_0.

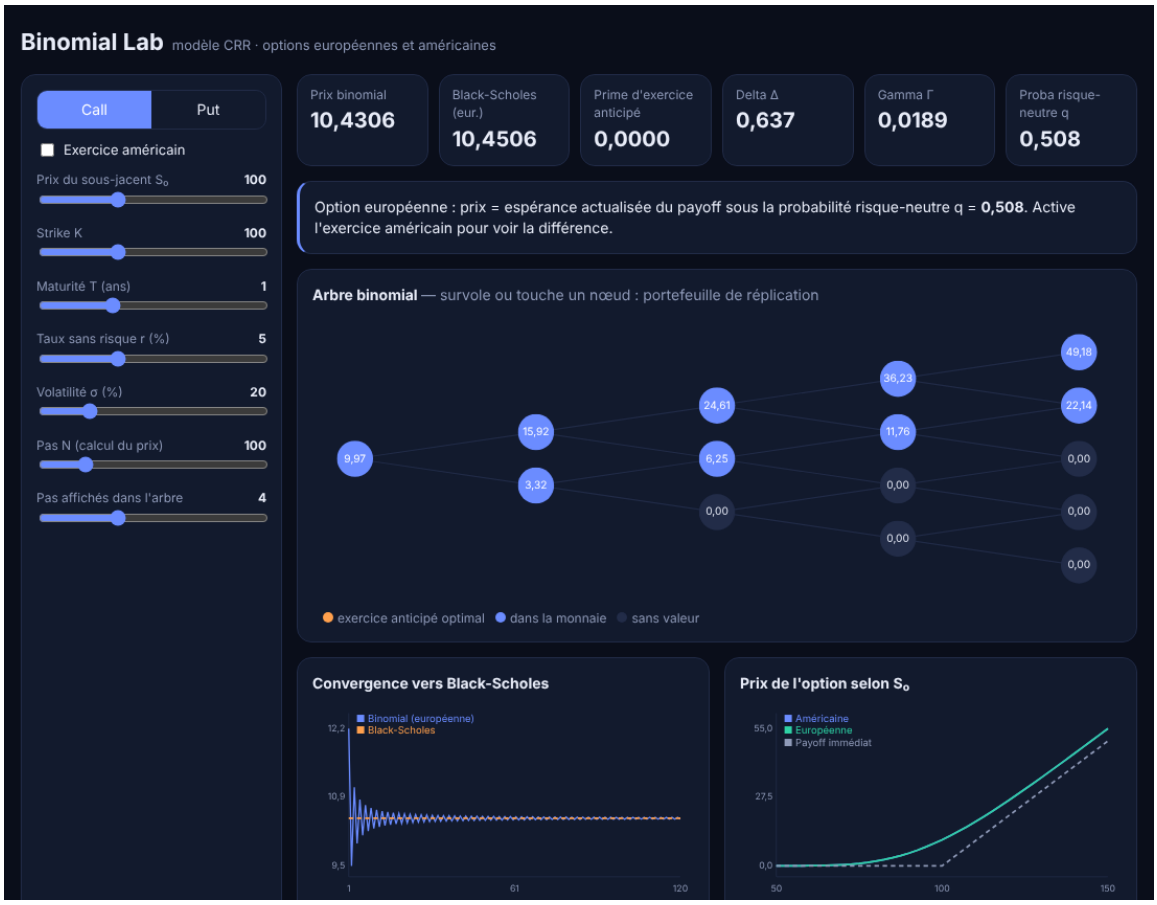


Figure 12: Interface : call européen, cas de référence. Prix binomial 10,4306 ($N = 100$) contre 10,4506 pour Black–Scholes.

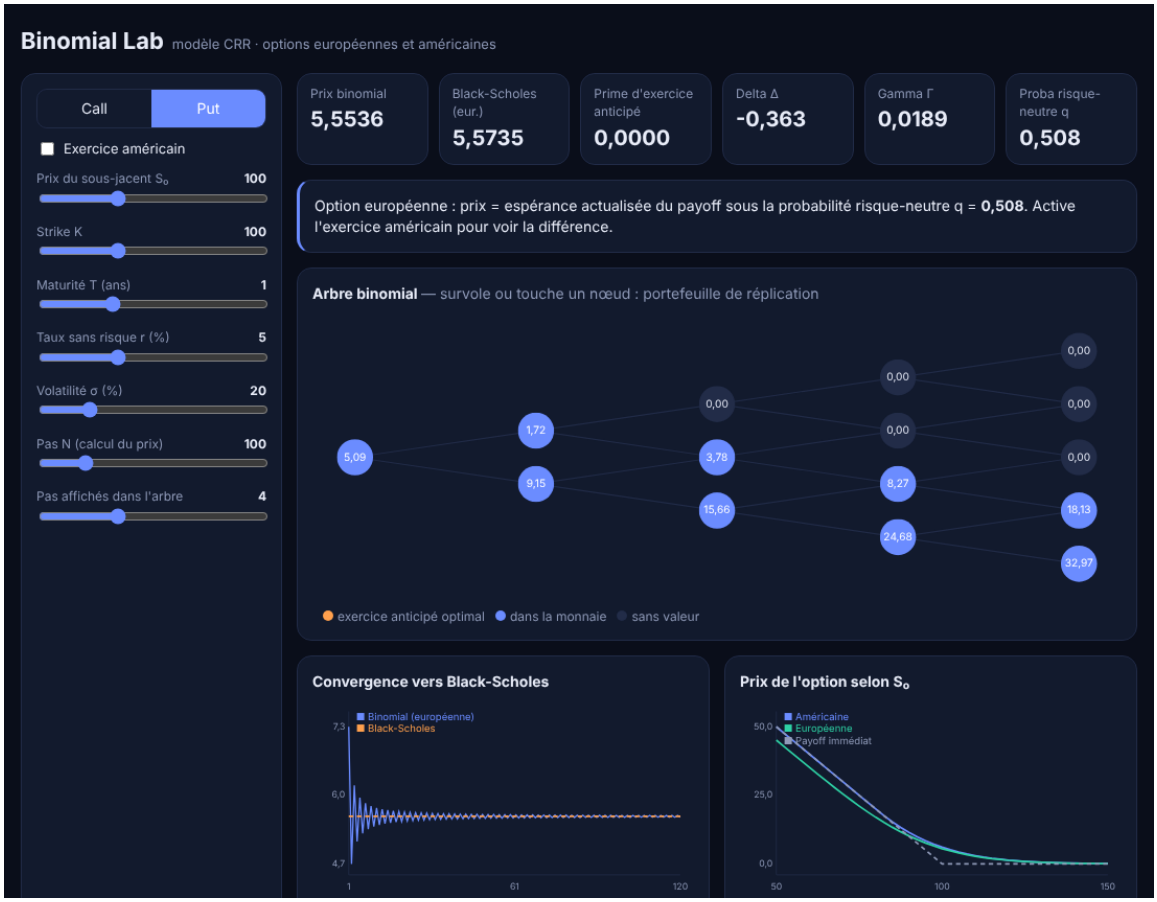


Figure 13: Interface : put européen, cas de référence. Prix binomial 5,5536 ($N = 100$) contre 5,5735 pour Black–Scholes.

On remarque sur les deux arbres la logique du payoff : pour le call, les nœuds hauts sont dans la monnaie et les nœuds bas valent zéro ; pour le put, c'est l'inverse. Sur les graphiques de convergence, le prix oscille autour de la ligne de Black–Scholes avant de se stabiliser, comme au chapitre 6.



Strike	Bid	Ask	Mid	Modèle	Écart	Vol. impl.
275,0	65,65	67,45	66,55	66,30	-0,25	144,3 %
280,0	60,65	62,95	61,80	61,30	-0,50	149,8 %
285,0	55,60	58,00	56,80	56,30	-0,50	138,4 %
290,0	50,30	51,75	51,02	51,30	0,27	–
295,0	46,10	47,55	46,83	46,30	-0,53	117,3 %
300,0	40,65	42,10	41,38	41,30	-0,07	77,2 %
302,5	38,35	39,35	38,85	38,80	-0,05	69,1 %
305,0	35,20	36,75	35,98	36,30	0,33	–
307,5	32,75	35,15	33,95	33,80	-0,15	71,0 %
310,0	30,90	33,00	31,95	31,30	-0,65	88,0 %
312,5	27,70	30,30	29,00	28,80	-0,20	64,7 %
315,0	25,50	26,85	26,18	26,30	0,13	–
317,5	22,45	24,40	23,42	23,80	0,38	–
320,0	20,30	22,35	21,33	21,31	-0,02	35,5 %
322,5	18,40	20,10	19,25	18,81	-0,44	53,4 %
325,0	16,25	16,80	16,52	16,31	-0,22	40,7 %

Figure 14: Interface : comparaison modèle/marché pour les calls Apple (AAPL), échéance du 14 octobre 2026, données de l'instantané du 12 octobre 2026 (Yahoo Finance). À gauche, prix de marché (milieu offre–demande) et prix du modèle ; à droite, volatilité implicite de notre modèle et celle publiée par Yahoo. Pour ces options cotées, l'interface utilise la variante américaine de l'arbre (voir section 6.7).

Annexe B : Code Python du modèle binomial

```python
"""
Modèle binomial (arbre recombinant) : options européennes.
Code : numpy sert uniquement aux calculs de base,
scipy.stats.norm uniquement pour la formule de Black-Scholes (référence).

Notations du cours : R = 1 + r (facteur de capitalisation par période),
u, d facteurs de hausse/baisse, q probabilité risque-neutre.
"""
import numpy as np
from math import exp, sqrt, log
from scipy.stats import norm


# ------------------------------------------------------------------ paramètres
def crr_params(r, sigma, T, N):
    """Calibrage de Cox-Ross-Rubinstein : u = e^(sigma sqrt(dt)), d = 1/u,
    R = e^(r dt) (taux continu converti en facteur par période)."""
    dt = T / N
    u = exp(sigma * sqrt(dt))
    return u, 1.0 / u, exp(r * dt)


def proba_risque_neutre(u, d, R):
    """q = (R - d)/(u - d). Exige l'absence d'arbitrage : d < R < u."""
    if not (d < R < u):
        raise ValueError("Arbitrage possible : il faut d < R < u "
                         f"(d={d:.4f}, R={R:.4f}, u={u:.4f})")
    return (R - d) / (u - d)


def payoff(S, K, option):
    """h(S) : max(S-K,0) pour un call, max(K-S,0) pour un put."""
    return np.maximum(S - K, 0.0) if option == "call" else np.maximum(K - S, 0.0)


# ------------------------------------------------- pricing (version rapide)
def prix_binomial(S0, K, N, u, d, R, option="call"):
    """Prix à t=0 par backward induction (vecteurs : O(N) mémoire).
    Au pas k, le noeud j (j = nombre de hausses) vaut S0 * u^j * d^(k-j)."""
    q = proba_risque_neutre(u, d, R)
    j = np.arange(N + 1)
    V = payoff(S0 * u**j * d**(N - j), K, option)          # valeurs à maturité
    for k in range(N - 1, -1, -1):
        V = (q * V[1:k + 2] + (1 - q) * V[0:k + 1]) / R     # continuation
    return float(V[0])


# ---------------------------------------------- pricing (arbre complet stocké)
def arbre_complet(S0, K, N, u, d, R, option="call"):
    """Retourne l'arbre complet : prix S, valeurs V et delta.
    Les tableaux sont indexés [k, j] (k = pas, j = nb de
    hausses), seules les cases j <= k sont utilisées."""
    q = proba_risque_neutre(u, d, R)
    S = np.full((N + 1, N + 1), np.nan)
    V = np.full((N + 1, N + 1), np.nan)
    delta = np.full((N + 1, N + 1), np.nan)
    for k in range(N + 1):
        j = np.arange(k + 1)
        S[k, :k + 1] = S0 * u**j * d**(k - j)
    V[N, :N + 1] = payoff(S[N, :N + 1], K, option)
    for k in range(N - 1, -1, -1):
```

```
        cont = (q * V[k + 1, 1:k + 2] + (1 - q) * V[k + 1, 0:k + 1]) / R
        # delta (cours, ch. 3.6) : (V(uS) - V(dS)) / ((u - d) S)
        delta[k, :k + 1] = (V[k + 1, 1:k + 2] - V[k + 1, 0:k + 1]) / \
                           (S[k + 1, 1:k + 2] - S[k + 1, 0:k + 1])
        V[k, :k + 1] = cont
    return S, V, delta


# ------------------------------------------------------------ Black-Scholes
def black_scholes(S0, K, T, r, sigma, option="call"):
    """Formule fermée (référence de validation)."""
    d1 = (log(S0 / K) + (r + sigma**2 / 2) * T) / (sigma * sqrt(T))
    d2 = d1 - sigma * sqrt(T)
    if option == "call":
        return S0 * norm.cdf(d1) - K * exp(-r * T) * norm.cdf(d2)
    return K * exp(-r * T) * norm.cdf(-d2) - S0 * norm.cdf(-d1)


# ------------------------------------------------------ raccourci CRR (r, sigma)
def prix_crr(S0, K, T, r, sigma, N, option="call"):
    u, d, R = crr_params(r, sigma, T, N)
    return prix_binomial(S0, K, N, u, d, R, option)


# ------------------------------------------------------------------- tests
def tests():
    # 1) exemple du cours : S0=100, r=5%, u=1.1, d=0.9, K=100, 1 période
    u, d, R = 1.1, 0.9, 1.05
    assert abs(proba_risque_neutre(u, d, R) - 0.75) < 1e-12
    c = prix_binomial(100, 100, 1, u, d, R, "call")
    p = prix_binomial(100, 100, 1, u, d, R, "put")
    assert abs(c - 7.142857) < 1e-5 and abs(p - 2.380952) < 1e-5
    _, _, dl = arbre_complet(100, 100, 1, u, d, R, "call")
    assert abs(dl[0, 0] - 0.5) < 1e-12
    # 2) parité call-put européenne : C - P = S0 - K / R^n
    for n in (1, 5, 50):
        c = prix_binomial(100, 95, n, u, d, 1.01, "call")
        p = prix_binomial(100, 95, n, u, d, 1.01, "put")
        assert abs((c - p) - (100 - 95 / 1.01**n)) < 1e-9
    # 3) cohérence des deux implémentations
    S, V, _ = arbre_complet(100, 100, 200, *crr_params(.05, .2, 1, 200), "put")
    assert abs(V[0, 0] - prix_crr(100, 100, 1, .05, .2, 200, "put")) < 1e-9
    # 4) bornes de non-arbitrage : max(S0 - K e^{-rT}, 0) <= C <= S0
    c = prix_crr(100, 100, 1, .05, .2, 200)
    assert max(100 - 100 * exp(-.05), 0) <= c <= 100
    # 5) convergence vers Black-Scholes
    assert abs(prix_crr(100, 100, 1, .05, .2, 2000) - black_scholes(100, 100, 1, .05, .2)) < 5e
    -3
    print("Tous les tests passent.")


if __name__ == "__main__":
    tests()
```

Annexe C : Code Python du modèle trinomial

```python
"""
Modèle trinomial (arbre recombinant) : options européennes.
numpy sert au calcul vectoriel, scipy.stats.norm à Black-Scholes (référence).

A chaque pas, le sous-jacent est multiplié par u (hausse), 1 (stable) ou
d = 1/u (baisse). Au pas k, le noeud j (j = -k..k) vaut S0 * u^j.
"""
import numpy as np
from math import exp, sqrt
from binomial import payoff, black_scholes


# ------------------------------------------------------------------ paramètres
def trinomial_params(r, sigma, T, N, lam=sqrt(3)):
    """Calibrage sur la volatilité (Boyle 1986, Kamrad-Ritchken 1991) :
    u = e^(lam sigma sqrt(dt)), d = 1/u, R = e^(r dt).
    lam = 1 redonne l'arbre binomial CRR ; lam = sqrt(3) est le choix de Hull."""
    dt = T / N
    u = exp(lam * sigma * sqrt(dt))
    return u, 1.0 / u, exp(r * dt), 1.0 - 1.0 / lam**2      # u, d, R, pm


def probas_risque_neutre(u, d, R, pm):
    """On fixe pm (choix de calibrage : le marché est incomplet, q n'est pas
    unique), puis on impose la condition martingale E_Q[S1] = R S0 :
        pu + pm + pd = 1   et   pu u + pm + pd d = R."""
    pu = (R - pm - (1 - pm) * d) / (u - d)
    pd = 1.0 - pm - pu
    if not (0 < pu < 1 and 0 < pd < 1 and 0 <= pm < 1):
        raise ValueError(f"Probabilités hors de ]0,1[ : pu={pu:.4f}, pm={pm:.4f}, pd={pd:.4f}"
                         " (réduire le pas de temps ou changer lambda)")
    return pu, pm, pd


# ------------------------------------------------- pricing (version rapide)
def prix_trinomial(S0, K, N, u, d, R, pm, option="call"):
    """Prix à t=0 par récurrence rétrograde. Au pas k il y a 2k+1 noeuds,
    stockés de j=-k (bas) à j=+k (haut)."""
    pu, pm, pd = probas_risque_neutre(u, d, R, pm)
    j = np.arange(-N, N + 1)
    V = payoff(S0 * u**j, K, option)                        # maturité
    for k in range(N - 1, -1, -1):
        # pour le noeud j du pas k : hausse -> j+1, stable -> j, baisse -> j-1
        V = (pu * V[2:] + pm * V[1:-1] + pd * V[:-2]) / R    # continuation
    return float(V[0])


# ------------------------------------------- pricing (arbre complet stocké)
def arbre_trinomial(S0, K, N, u, d, R, pm, option="call"):
    """Retourne S et V, indexés [k, j+N]."""
    pu, pm, pd = probas_risque_neutre(u, d, R, pm)
    S = np.full((N + 1, 2 * N + 1), np.nan)
    V = np.full_like(S, np.nan)
    for k in range(N + 1):
        j = np.arange(-k, k + 1)
        S[k, N - k:N + k + 1] = S0 * u**j
    V[N] = payoff(S[N], K, option)
    for k in range(N - 1, -1, -1):
        a, b = N - k, N + k + 1
        cont = (pu * V[k + 1, a + 1:b + 1] + pm * V[k + 1, a:b] + pd * V[k + 1, a - 1:b - 1]) /
    R
```

```
        V[k, a:b] = cont
    return S, V


def greeks_trinomial(S0, K, T, r, sigma, N, option="call", lam=sqrt(3)):
    """Delta et gamma à t=0 lus sur les trois noeuds du premier pas :
    avantage du trinomial, le gamma est disponible dès t = dt."""
    u, d, R, pm = trinomial_params(r, sigma, T, N, lam)
    S, V = arbre_trinomial(S0, K, N, u, d, R, pm, option)
    Sd, Sm, Su = S[1, N - 1:N + 2]
    Vd, Vm, Vu = V[1, N - 1:N + 2]
    delta = (Vu - Vd) / (Su - Sd)
    gamma = ((Vu - Vm) / (Su - Sm) - (Vm - Vd) / (Sm - Sd)) / ((Su - Sd) / 2)
    return delta, gamma


def prix_tri(S0, K, T, r, sigma, N, option="call", lam=sqrt(3)):
    u, d, R, pm = trinomial_params(r, sigma, T, N, lam)
    return prix_trinomial(S0, K, N, u, d, R, pm, option)


# ---------------------------------------------------------------------- tests
def tests():
    from binomial import prix_crr
    # 1) lambda = 1 : on retrouve exactement l'arbre binomial CRR
    for opt in ("call", "put"):
        assert abs(prix_tri(100, 100, 1, .05, .2, 50, opt, lam=1.0)
                   - prix_crr(100, 100, 1, .05, .2, 50, opt)) < 1e-10
    # 2) exemple du rapport : u=1.25, d=0.8, R=1.05, pm=1/2 -> pu=1/3, pd=1/6
    pu, pm, pd = probas_risque_neutre(1.25, 0.8, 1.05, 0.5)
    assert abs(pu - 1 / 3) < 1e-12 and abs(pd - 1 / 6) < 1e-12
    # 3) parité call-put exacte dans l'arbre
    u, d, R, pm = trinomial_params(.05, .2, 1, 100)
    c = prix_trinomial(100, 95, 100, u, d, R, pm, "call")
    p = prix_trinomial(100, 95, 100, u, d, R, pm, "put")
    assert abs((c - p) - (100 - 95 / R**100)) < 1e-9
    # 4) cohérence des deux implémentations
    S, V = arbre_trinomial(100, 100, 100, u, d, R, pm, "put")
    assert abs(V[0, 100] - prix_trinomial(100, 100, 100, u, d, R, pm, "put")) < 1e-10
    # 5) gamma lu au premier pas proche de Black-Scholes (0.01876)
    _, g = greeks_trinomial(100, 100, 1, .05, .2, 500, lam=sqrt(1.5))
    assert abs(g - 0.018762) < 1e-4
    # 6) convergence vers Black-Scholes
    assert abs(prix_tri(100, 100, 1, .05, .2, 1000) - black_scholes(100, 100, 1, .05, .2)) < 5e
    -3
    print("Tous les tests trinomiaux passent.")


if __name__ == "__main__":
    tests()
```

Bibliographie

[1] R. Elie, I. Kharroubi, *Calcul stochastique appliqué à la finance*, support de cours (chapitres 2 à 4 : modèle binomial à une et n périodes ; chapitre 6 : modèle de Black–Scholes).

[2] S. Shreve, *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*, Springer, 2004.

[3] J. Hull, *Options, Futures, and Other Derivatives*, Pearson (chapitres sur les arbres binomiaux et les arbres trinomiaux).

[4] J. Cox, S. Ross, M. Rubinstein, *Option Pricing: A Simplified Approach*, Journal of Financial Economics 7(3), 1979.

[5] P. Boyle, *Option Valuation Using a Three-Jump Process*, International Options Journal 3, 1986, p. 7–12.

[6] B. Kamrad, P. Ritchken, *Multinomial Approximating Models for Options with k State Variables*, Management Science 37(12), 1991, p. 1640–1652.

[7] M. Brennan, E. Schwartz, *Finite Difference Methods and Jump Processes Arising in the Pricing of Contingent Claims: A Synthesis*, Journal of Financial and Quantitative Analysis 13(3), 1978.

[8] F. Black, M. Scholes, *The Pricing of Options and Corporate Liabilities*, Journal of Political Economy 81(3), 1973.

[9] R. Merton, *Theory of Rational Option Pricing*, Bell Journal of Economics and Management Science 4(1), 1973.

[10] D. Leisen, M. Reimer, *Binomial Models for Option Valuation: Examining and Improving Convergence*, Applied Mathematical Finance 3(4), 1996.

[11] F. Longstaff, E. Schwartz, *Valuing American Options by Simulation: A Simple Least-Squares Approach*, Review of Financial Studies 14(1), 2001.